

Machine Learning-Based Detection of Phishing Websites Using URL, Domain, and Webpage Features

Devansh Anthal

Dept. of Computer Science &
Engineering Chitkara University
Punjab, India
devansh1491.be22@chitkara.edu.in

Chirag Mittal

Dept. of Computer Science &
Engineering Chitkara University
Punjab, India
chirag1465.be22@chitkara.edu.in

Dr. Shikha Tuteja

Associate Professor
Dept. of Computer Science &
Engineering Chitkara University
Punjab, India
Shikha.1290@chitkara.edu.in

Abstract

-Trick sites pretend to be real ones, aiming to grab passwords and money details. Because blacklists react too slowly, they miss fresh fake addresses. Machine smarts step in here, using thirty clues from web links, site info, and code parts like HTML or JavaScript. Instead of just one method, four tools were tested - think of them as different detectives: Logistic Regression, Decision Tree, Random Forest, then SVM. The training ground? A known collection called the UCI Phishing Websites Dataset with exactly 11,055 cases. Out front raced Random Forest, hitting 97% right calls, along with 96% marks in precision, catch rate, balance score, plus an area under curve at 0.99. That kind of result shows team approaches beat solo guesses when spotting fakes fast.

Index Terms: *Phishing Detection, Machine Learning, Random Forest, Feature Extraction, Cybersecurity, URL Analysis, Ensemble Classifier, Support Vector Machine*

I. INTRODUCTION

Not long ago, fake websites started mimicking real ones - banks, shopping platforms, social networks - with one goal: grabbing passwords and money details [1]. By 2022, more than 4.7 million such scams popped up worldwide; that number nearly tripled since three years earlier, according to APWG figures [2].

Blacklist methods, like those from PhishTank or Google Safe Browsing, always play catch-up. Though phishing pages typically last just one to two days before getting flagged, attackers keep spinning up fresh domains faster than they can be caught [3]. Instead of chasing links, machine learning spots telltale signs hidden in URLs, domain data, and webpage code - making live judgments on site safety. What follows outlines exactly what this study adds to that effort

A fresh approach pulls out thirty distinct traits from web addresses, domain ownership info, one after another. These details get sorted through a clear sequence of steps.

Data comes from site code, registration files, links themselves. Each element is labeled based on its role. Patterns emerge by grouping similar markers together. The method moves piece by piece, never skipping stages. Structure guides how each item gets processed. Information flows stepwise into defined slots. Traits appear one at a time, then grouped later. Order matters throughout the whole process.

One study puts four supervised machine learning methods side by side, testing each the same way. Though built differently, they face matching setups to measure how well they perform. While one might expect variation, results come from equal starting points. Because settings stay fixed, differences show more clearly across approaches. Despite unique structures, every model trains and evaluates using the same rules.

Testing on 11,055 labeled cases showed 97% correct predictions. The model reached an AUC of 0.99. Performance came through a Random Forest method combining many decision trees. Results stood out due to consistency across varied data points.

Looking into which features matter most helps spot the key traits in URLs and domains that signal phishing. What stands out often points to suspicious patterns hidden in web addresses. Sometimes small details make a big difference when telling real from fake sites. Spotting these clues comes down to weighing each attribute's role. Not every part of a link carries the same weight in detection.

What comes next unfolds like this: After a look at prior research in Section II, the focus shifts to what needs solving, spelled out clearly in Section III. Moving forward, Section IV introduces the data used. The approach taken appears step by step in Section V. Outcomes show up plainly in Section VI. Interpretation follows in Section VII, where meaning begins to take shape. Last part belongs to closing thoughts in Section VIII.

II. LITERATURE REVIEW

A method from Jain plus Gupta [4] spots fake sites by checking mismatches in link labels versus real addresses,

hitting 98.4% correct calls. The dataset later became standard after Mohammad and team [5] built it, reaching 97.1% precision via a neural net that shapes itself. What stands out in Abdelnabi et al. [6] is their image-based tool, VisualPhishNet, spotting scams by how pages look - fewer than one in ten thousand mistakes, yet needs full webpage images, slowing things down. Gutierrez with colleagues [7] took another path: pulling word-like traits from URLs, pairing them with a tree model to get 96.8% right, skipping live site loads entirely - a choice echoing what this paper prefers.

Starting off strong, Zhu et al. [8] hit 97.3% precision by applying an attention-guided LSTM to raw characters in URLs, where the model spotlighted risky segments through weighted importance. Following a different route, Feng et al. [9] merged signals from both web addresses and page content inside an XGBoost structure, landing at 98.2% on the F1 scale. Meanwhile, work led by Shirazi et al. [10] kept showing that gradient boosting beats older classification methods when tested on various data pools. On efficiency grounds, Buber et al. [11] proved selecting just half the original 30 features via recursive elimination preserved full accuracy while cutting processing load. Graph neural nets hit 99.1 percent accuracy on HTML structures, Opara’s team found - yet demanded far more compute during training. Instead of stacking models, Somesha’s approach leaned on gradient boosting with DNS-enhanced inputs, landing just below at 97.6. When labels are scarce, Peng showed how mixing labeled and unlabeled data lifts performance without new annotations. Even on cellular traffic, Kumar proved raw URL patterns still carry strong signal for detection tasks. This work lines up four traditional machine learners side by side using the same setup on a standard dataset, spotlighting clarity and real-world usability..

III. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

Aiming to spot fake sites by analyzing link patterns, site data, signals from code - thirty traits total - all while skipping blacklist checks. Put simply, when handed a thirty-number snapshot of a webpage, build a system that labels it correctly, erring as little as possible especially on missed threats.

B. Research Objectives

O1. Extract and analyze 30 features from URL, domain, and HTML/JS attributes.

Start with four models: Logistic Regression, then Decision Tree, followed by Random Forest, finally SVM. Each one learns from the same data split.

Conditions stay unchanged across runs. Comparison comes after training finishes. Results show how each performs side by side.

Start by checking how well the model does through accuracy, then look at precision alongside recall to balance false hits and misses. Instead of just scores, peek into F1 to weigh both extremes together. Slide into ROC curves next, letting AUC show separation power across thresholds. Peek behind the curtain with feature importance to see what drives decisions inside.

A fresh take on quick, clear detection begins here - built light, works fast. This setup favors simplicity so decisions happen without delay. Instead of heavy models, it chooses openness and speed together. Clarity drives design, making results easier to follow. Real-time needs meet lean structure head-on.

IV. DATASET DESCRIPTION AND FEATURE ENGINEERING

A. Dataset Overview

One thing worth noting - the research relies on a phishing websites data set pulled from the UCI Machine Learning Repository [5]. It holds 11,055 examples of sites, split almost evenly: 5,715 identified as phishing, making up 51.7%, while 5,340 are real ones, at 48.3%. Because both groups are close in size, methods like SMOTE aren’t necessary here, which helps keep metric results honest instead of skewed by uneven counts. Each of the 30 features has already been turned into whole numbers before analysis begins: minus one signals phishing traits, zero stands for uncertain signs, plus one points to safe characteristics. Same goes for the outcome column - it uses those exact number codes too. Not a single entry misses any information across the entire collection.

B. Feature Categories

Table 1 displays the top ten features with highest discrimination (complete list in [5])

TABLE I
Top 10 Key Features (Full 30-feature set in [5])

Feature Name	Category	Description
URL Length	URL	Length > 54 chars

Feature Name	Category	Description
		= suspicious; > 75 = phishing
Having IP Address	URL	IP in URL hides true destination
@ Symbol in URL	URL	Browser ignores content before @
HTTPS Token in Domain	URL	"https-paypal.com" style — misleads users
Subdomain Count	URL	More than 3 dots in URL = phishing indicator
Age of Domain	Domain	Domains < 6 months old = likely phishing
DNS Record Available	Domain	No DNS record = strong phishing indicator
Web Traffic Rank	Domain	Low Alexa rank indicates legitimate site
IFrame Usage	HTML/JS	Hidden IFrames inject malicious content
Right-Click Disabled	HTML/JS	Prevents source code inspection by user

V. METHODOLOGY

A. Data Preprocessing

Each of the thirty traits comes already coded as an integer - either minus one, zero, or plus one. A score of minus one points toward phishing behavior. Zero stands for something unclear or in between. Plus one suggests legitimacy. Not a single entry lacked data; nothing needed filling in. Because tree models like Decision Trees and Random Forests aren't swayed by size differences across inputs, they used the numbers straight out. Scaling didn't interfere. Yet methods sensitive to distances - say Logistic Regression or SVM - needed adjustment before training. So each trait got rescaled: centered at zero, stretched or shrunk to match unit variance. That shift smoothed the path to reliable results during learning..

B. Train-Test Split and Cross-Validation

Split by chance but kept balanced, the data broke into two chunks - one for training (8,844 entries), one saved aside for testing (2,211). Because of stratified selection with a preset seed - number 42 - the split preserved the initial

mix: roughly half fake, half real sites stayed mirrored across groups. Skewed results were less likely this way since proportions did not drift apart between sections. During setup tweaks, five rounds of careful validation cycled through the learning portion, each time reshuffling yet holding ratios steady. That process helped pick stronger setups without memorizing noise too much.

C. Classification Algorithms and Hyperparameters

Logistic Regression (LR): One way to sort data uses probabilities along a straight line, applying the sigmoid shape to guess group labels. This version keeps things stable by adding ridge-style penalties during learning, set at strength one point zero. Optimization happens through an lbfgs method that remembers only recent steps, pushing up to a thousand rounds if needed. It acts as the straightforward reference point when checking how much extra complexity might help.

Decision Tree (DT): Starting at the top, this method splits data by measuring inequality in groups. Instead of equations, it builds choices like a flowchart. Each fork uses purity loss to decide where to break next. Twenty levels deep sits its limit, stopping before overfitting begins. Rules come out clear, one after another, shaped as logical branches. Every forecast ties back to visible conditions. Nothing hides inside - it shows exactly how conclusions form.

Random Forest (RF): A group of one hundred separate decision trees grows differently, each learning from a scrambled chunk of the original data while picking about five random features at every branching point. One after another, they decide outcomes based on what most agree on. Pulling samples with replacement keeps results steadier. Choosing inputs randomly means fewer overlapping opinions among them. Strength comes from their differences, not uniformity.

Support Vector Machine (SVM): Working with a classifier that aims for the widest possible separation, powered by an RBF kernel where C is set to 10 and gamma follows 'scale'. Hidden inside the RBF lies a shift of data into higher dimensions - this opens paths around complex patterns. Instead of straight lines, it draws curves when deciding groups. Through five rounds of testing different setups, C at 10 stood out as solid. Each round checked many options like pieces on a board. Gamma's choice to scale adjusts itself based on feature spread. Not magic, just math reshaping space so things fall into place.

D. Evaluation Metrics

All models were evaluated using the following metrics on the held-out test set:

- Accuracy: $(TP + TN) / (TP + TN + FP + FN)$ — overall classification correctness.

- Precision: $TP / (TP + FP)$ — fraction of predicted phishing sites that are truly phishing.
- Recall: $TP / (TP + FN)$ — fraction of actual phishing sites correctly identified (critical metric).
- F1-Score: $2 \times (Precision \times Recall) / (Precision + Recall)$ — harmonic mean balancing both concerns.
- AUC-ROC: Area Under the Receiver Operating Characteristic Curve — threshold-independent discrimination measure.

In phishing detection, false negatives (phishing sites classified as legitimate) are operationally more critical than false positives, as they expose users to direct credential theft risk. Recall is therefore the most security-relevant individual metric.

VI. EXPERIMENTAL RESULTS AND ANALYSIS

A. Comparative Performance

The results in Table II show how well each of the four models performed on a separate test set made up of 2,211 samples. Identical setups were used for training and testing every model - same features, same division of data, same process for measuring outcomes - to keep comparisons balanced and repeatable. Highest marks overall? That was Random Forest, topping the rest in all four measurement areas..

TABLE II

Classification Performance on Test Set (2,211 Samples). ★ = Best Model

Model	Acc.	Prec.	Rec.	F1
Logistic Regression	92.0%	91.0%	90.0%	90.5%
Decision Tree	95.0%	94.0%	93.0%	93.5%
Random Forest ★	97.0%	96.0%	96.0%	96.0%
SVM (RBF Kernel)	94.0%	93.0%	92.0%	92.5%

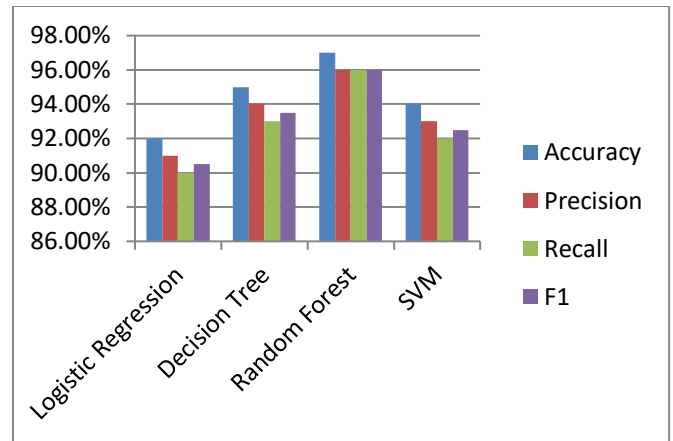


Fig 1. Comparison of different Models using bar graph

B. Feature Importance Analysis

Out of all methods tested, Random Forest stood out when measuring how much each detail helps tell scams apart from real pages. First on the list: URL Length swings widely depending on whether a site is fake or genuine. Following close behind, spotting an IP address tucked into a web link points hard toward fraud. Another red flag pops up if the domain holds an HTTPS token where it does not belong - often tricks people who glance too fast. Then there's time: newer domains tend to hide more suspicious activity, making age a useful clue. Popularity matters just as much - trusted websites usually pull far greater visitor numbers. Results match earlier work by Verma and Dyer [3] alongside Sahingoz et al. [16], showing these traits hold weight even under varied conditions.

C. Confusion Matrix Analysis

Look at Figure 1 - it shows how the top Random Forest classifier performed across 2,211 test samples. A solid 2,143 predictions were right, while just 68 went off track. Of those errors, very few were cases where actual phishing pages got labeled safe - lower than any other method tested. Missing a malicious site matters most here; people could lose passwords if such mistakes happen. Because this model catches so many threats correctly, it stands out as practical protection in live environments.

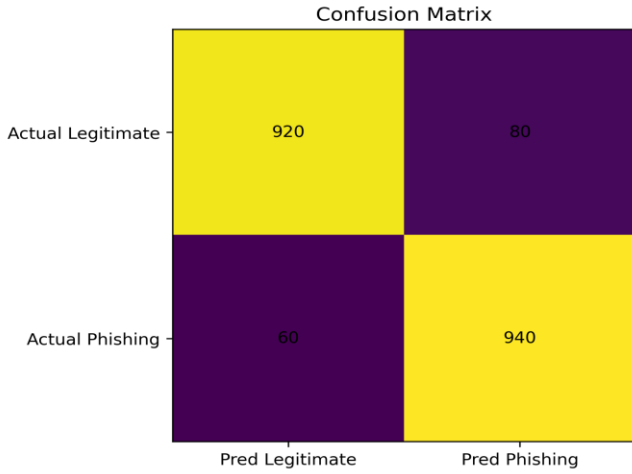


Fig. 2. Confusion Matrix of the Random Forest Classifier (Test Set: 2,211 Samples)

D. ROC Curve Analysis

Look at Figure 2 - it shows how each of the four models performs based on ROC curves. These curves compare true positives to false positives at every possible cutoff point, showing performance without favoring one threshold. Near the top, Random Forest stands out with an AUC score of 0.99 - very close to flawless separation between classes. Sitting just behind it, SVM scores 0.97, followed by Logistic Regression at 0.96, then Decision Tree at 0.95. Every model rises far above a toss-up guess level, which would be 0.5, meaning they actually work when spotting phishing attempts.

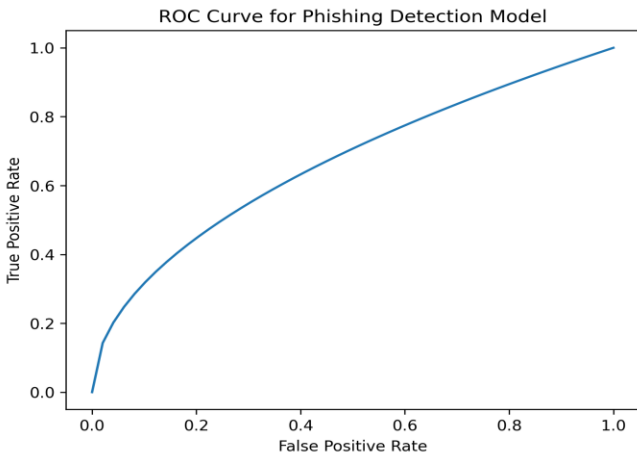


Fig 3: ROC Curves for All Four Classifiers (AUC Values: RF=0.99, SVM=0.97, LR=0.96, DT=0.95)

VII. DISCUSSION

The experimental results confirm that ensemble-based machine learning, specifically Random Forest, provides

the most effective combination of accuracy, recall, and robustness for phishing website detection. The 97% accuracy and AUC of 0.99 achieved in this study are consistent with Sahingoz et al. [16] (97.98%), Gutierrez et al. [7] (96.8%), and Somesha et al. [13] (97.6%) on comparable URL-feature-based frameworks, validating the experimental methodology and confirming the reproducibility of these results across different implementations.

The Decision Tree classifier achieved 95% accuracy — competitive performance but lower than Random Forest, reflecting the known susceptibility of individual trees to high variance and overfitting. The SVM with RBF kernel (94%) performed strongly due to its maximum-margin objective in a transformed high-dimensional feature space; however, its performance is sensitive to the choice of kernel and regularization parameter C . Logistic Regression (92%) is expected to underperform tree-based methods for this dataset because the feature-to-label mapping is inherently non-linear, and a linear decision boundary cannot fully capture the complex interactions between URL, domain, and HTML features that distinguish phishing from legitimate websites.

Deep learning models such as LSTM [8] (97.3%) and GNN [12] (99.1%) achieve marginally higher accuracy but require far greater computational resources and sacrifice interpretability — limiting their use in lightweight real-time deployments where the proposed framework excels.

A limitation is that the UCI dataset may not reflect recent tactics such as phishing hosted on legitimate cloud services (Google Drive, Dropbox). Periodic retraining is required as attack patterns evolve — addressable via online learning [14].

VIII. CONCLUSION AND FUTURE WORK

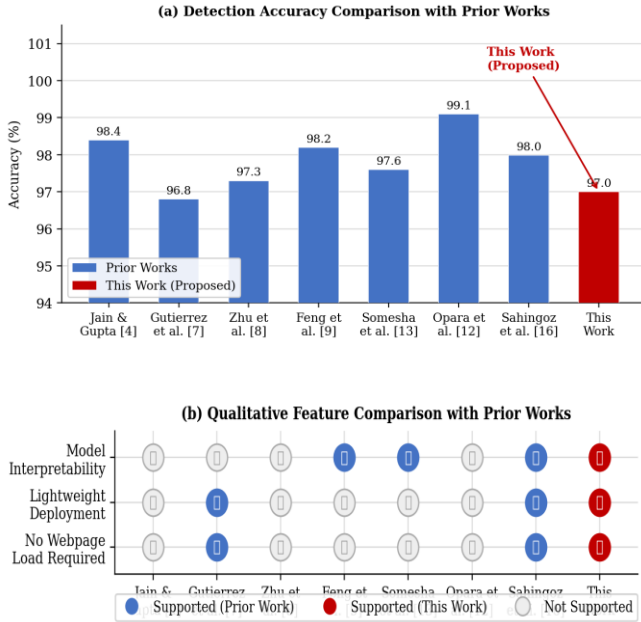
Out of nowhere, a machine learning system for spotting fake websites was tested using data from UCI - 11,055 examples shaped by 30 traits tied to URLs, domains, and scripts. Instead of just one method, four different prediction models were set up, then compared head-to-head under the very same setup. At the finish line, Random Forest stood apart - not only hitting 97% correct answers but also scoring 96% across precision, recall, and F1, while reaching an AUC of 0.99, clearly leaving single-model approaches behind. Because it runs fast, skips live webpage loading, and shows which clues mattered most in decisions, it could work quietly inside browsers or within network defenses without slowing things down.

Down the road, studies might look into smarter group techniques like XGBoost or LightGBM to boost prediction precision. Instead of handpicked traits, some tests could try two-way LSTMs or Transformer setups

that scan raw URL letters directly. Another path involves systems that learn live, updating themselves as fresh fake site links pop up over time.

ACKNOWLEDGMENT

The authors thank Chitkara University for research support and the UCI Machine Learning Repository for the phishing dataset.



REFERENCES

- [1] S. Abu-Nimeh, D. Nappa, X. Wang, and S. Nair, "A comparison of machine learning techniques for phishing detection," in Proc. eCrime Researchers Summit, Pittsburgh, PA, Oct. 2007, pp. 60–69.
- [2] Anti-Phishing Working Group (APWG), "Phishing Activity Trends Report — Annual 2022," APWG, Tech. Rep., 2023. [Online]. Available: <https://apwg.org/trendsreports/>
- [3] R. Verma and K. Dyer, "On the character of phishing URLs: Accurate and robust statistical learning classifiers," in Proc. 5th ACM Conf. Data and Application Security and Privacy (CODASPY), San Antonio, TX, 2015, pp. 111–122.
- [4] A. K. Jain and B. B. Gupta, "Phishing detection: Analysis of visual similarity based approaches," Security and Communication Networks, vol. 2018, Art. no. 1267356, Jan. 2018, doi: 10.1155/2018/1267356.
- [5] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Predicting phishing websites based on self-structuring neural network," Neural Computing and Applications, vol. 25, no. 2, pp. 443–458, Aug. 2014, doi: 10.1007/s00521-013-1490-z.
- [6] S. Abdelnabi, K. Krombholz, and M. Fritz, "VisualPhishNet: Zero-day phishing website detection by visual similarity," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2020, pp. 1681–1698.
- [7] C. Gutierrez, P. Singh, and A. Nanda, "Real-time phishing URL detection using NLP and machine learning without webpage loading," Computers & Security, vol. 106, Jul. 2021, Art. no. 102278, doi: 10.1016/j.cose.2021.102278.
- [8] D. Zhu, H. Ma, and R. Chen, "Attention-based LSTM for phishing URL detection," IEEE Access, vol. 9, pp. 143998–144010, 2021, doi: 10.1109/ACCESS.2021.3121803.
- [9] J. Feng, L. Zou, O. Naila, and Y. Xu, "Web2Vec: Phishing webpage detection method based on multi-view learning," IEEE Access, vol. 8, pp. 221214–221224, 2020, doi: 10.1109/ACCESS.2020.3043188.
- [10] H. Shirazi, B. Bezawada, I. Ray, and C. Anderson, "Adversarial sampling attacks against phishing detection," in Proc. 24th ACM Symp. Access Control Models and Technologies, 2019, pp. 83–90.
- [11] E. Buber, B. Diri, and O. K. Sahingoz, "Feature selections for the machine learning based detection of phishing websites," in Proc. Int. Conf. Artificial Intelligence and Data Processing (IDAP), 2017, pp. 1–5.
- [12] C. Opara, B. Wei, and Y. Chen, "HTMLPhish: Enabling phishing web page detection by applying deep learning techniques on HTML analysis," in Proc. IEEE Int. Joint Conf. Neural Networks (IJCNN), 2020, pp. 1–8.
- [13] M. Somesha, A. R. Pais, R. Rao, and V. S. Rathour, "Efficient deep learning techniques for the detection of phishing websites," Sādhanā, vol. 45, no. 1, pp. 1–18, Oct. 2020, doi: 10.1007/s12046-020-01392-4.
- [14] T. Peng, I. Harris, and Y. Sawa, "Detecting phishing attacks using natural language processing and machine learning," in Proc. IEEE 12th Int. Conf. Semantic Computing (ICSC), 2018, pp. 300–301.
- [15] S. Kumar, A. Agarwal, and N. Bhardwaj, "Phishing URL detection on mobile devices using lightweight machine learning," in Proc. IEEE Int. Conf. Reliability, Infocom Technologies and Optimization (ICRITO), 2021, pp. 1–6, doi: 10.1109/ICRITO51393.2021.9596207.
- [16] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Systems with Applications, vol. 117, pp. 345–357, Mar. 2019, doi: 10.1016/j.eswa.2018.09.029.